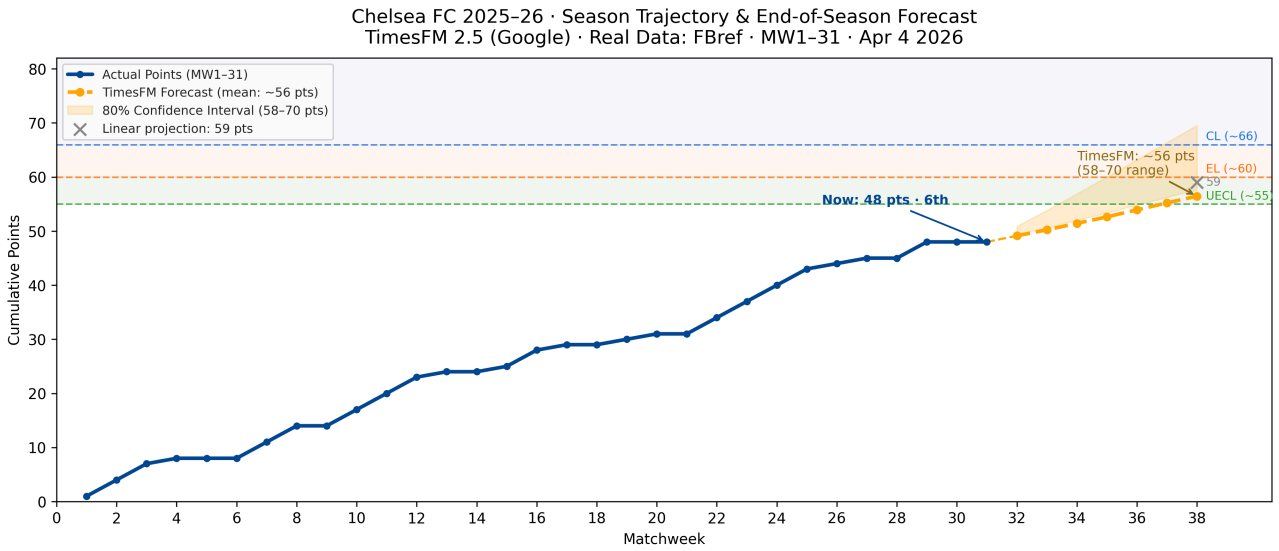
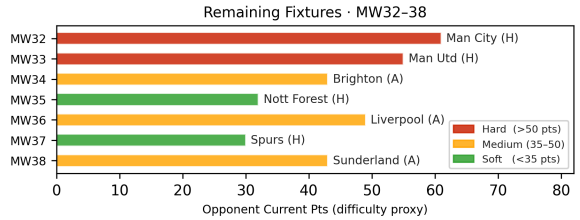
Chelsea FC 2025–26: TimesFM Forecast vs. Linear Projection

A live snapshot of Chelsea's end-of-season trajectory using Google's TimesFM 2.5 zero-shot time-series model.





Top 6 · MW31 Standings
(Proj = linear to 38 games)

Pos	Team	Pts	GD	Proj
1	Arsenal	70	+39	86
2	Man City	61	+32	77
3	Man Utd	55	+13	67
4	Aston Villa	54	+5	66
5	Liverpool	49	+8	60
6	**Chelsea**	**48**	**+15**	**59**

■ Blue = CL ■ Orange = EL ■ Green = UECL

The TL;DR: Why this matters

As of Matchweek 31, Chelsea sits in 6th place with 48 points. The simple, linear math says they are on pace for **59 points** by the end of the season.

But football isn't linear. Chelsea's recent form has flattened significantly (three losses in the last four matches), and the remaining schedule is heavily front-loaded with difficult fixtures (Man City and Man Utd back-to-back).

When we feed the actual match-by-match sequence into **TimesFM 2.5**—a 200-million parameter foundational model trained by Google on 100 billion time-points—it reads

that flattening momentum. The model projects a final tally of **56 points**, with an 80% confidence interval ranging from 58 to 70.

This creates a concrete, trackable tension for the final 7 matches: **Will Chelsea revert to their season-long mean (59 pts), or will the recent momentum decay hold true (56 pts)?** The TimesFM forecast suggests they will narrowly secure Conference League qualification (~55 pts threshold), but fall short of the Europa League (~60 pts).

Core Implementation: How the forecast works

The key to using a sequence model for cumulative points is to forecast the *per-match* points earned (0, 1, or 3) rather than the cumulative total directly. Forecasting a cumulative line that ends flat causes the model to predict the “level” rather than the growth.

Here is the core logic used to generate the forecast, using the `timesfm` library:

```python
import pandas as pd
import numpy as np
import timesfm
from timesfm.timesfm_2p5 import timesfm_2p5_torch

# 1. Load actual MW1-31 results
df = pd.read_csv('data/chelsea_real_2025_26.csv')
points_map = {'W': 3, 'D': 1, 'L': 0}
df['points_earned'] = df['result'].map(points_map)
current_pts = 48

# 2. Initialize TimesFM 2.5
model = timesfm_2p5_torch.TimesFM_2p5_200M_torch.from_pretrained(
    "google/timesfm-2.5-200m-pytorch"
)
model.compile(timesfm.ForecastConfig(
    max_context=512, max_horizon=10,
    normalize_inputs=True, use_continuous_quantile_head=True,
    force_flip_invariance=True, infer_is_positive=True,
fix_quantile_crossing=True,
))

# 3. Forecast the per-match points (horizon = 7 remaining matches)
pt_fc, qt_fc = model.forecast(
    horizon=7,
    inputs=[df['points_earned'].values.astype(float)]
)

# 4. Accumulate the forecast on top of the current baseline
per_match_mean = pt_fc[0]
projected_cumulative = current_pts + np.cumsum(per_match_mean)

print(f"Projected final points: {int(round(projected_cumulative[-1]))}")
# Output: Projected final points: 56
```

Next Steps for the Model

While TimesFM is powerful at reading sequence momentum, it is blind to context. The next iteration of this model will patch these gaps by injecting real-world signals:

1. **Fixture Difficulty Covariate:** Appending the opponent's current points total to the data, allowing the model to weight a loss to Man City differently than a loss to Sunderland.
2. **Home/Away Split:** Running parallel forecasts for home and away sequences, then recombining them based on the actual remaining schedule.
3. **Player Availability:** Manually flagging matchweeks where key players (e.g., Cole Palmer, Reece James) were unavailable, helping the model separate structural decline from temporary injury crises.

Data as of end of Matchweek 31 (April 4, 2026).